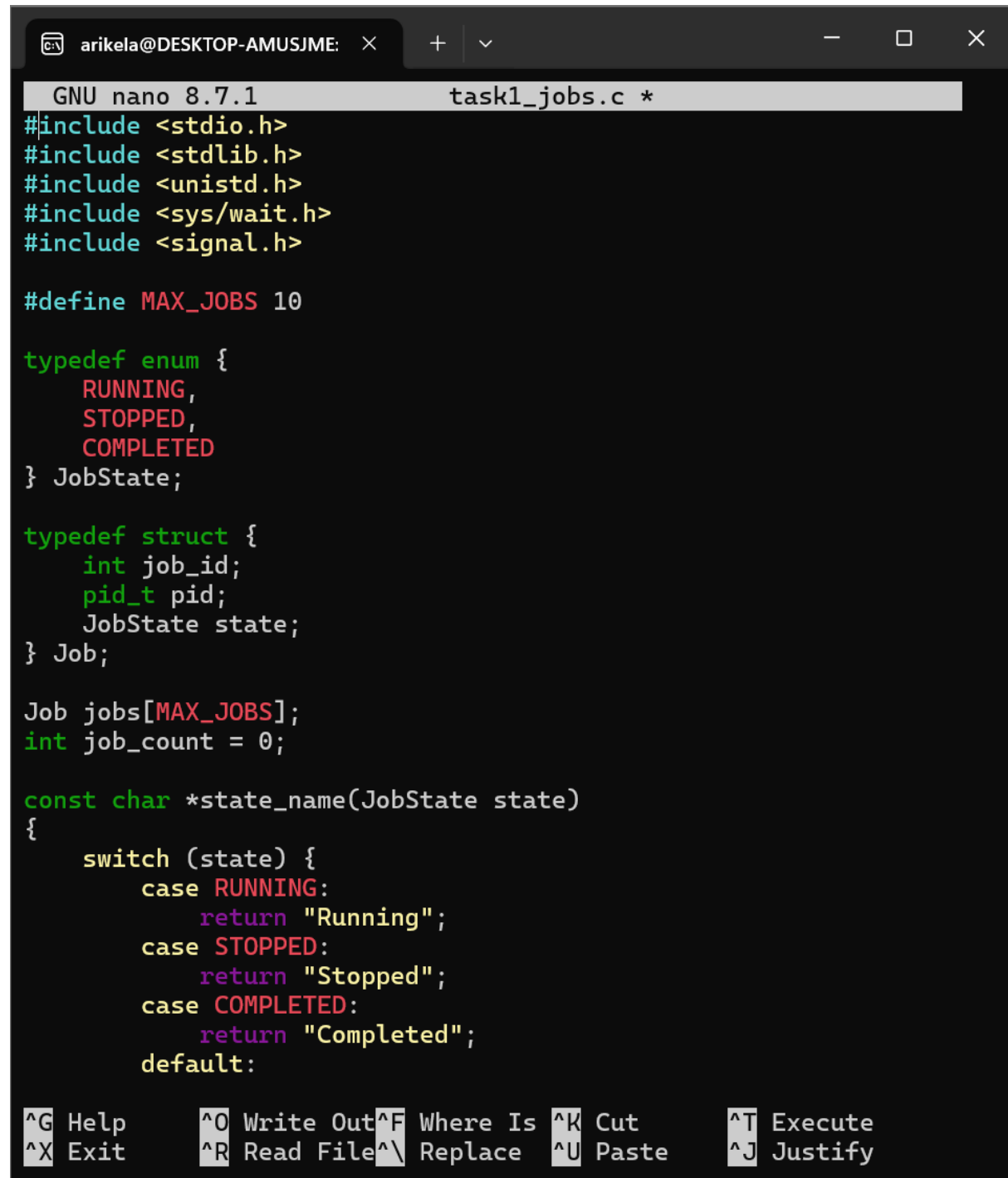


SKILL WEEK-9

TASK-1: To Implement how to List Active Jobs, Display Status Information, Retrieve Job [Ds, Format Output, Update State Information, Test Job Listings

Ans:

task1_jobs.c:



```
GNU nano 8.7.1 task1_jobs.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX_JOBS 10

typedef enum {
    RUNNING,
    STOPPED,
    COMPLETED
} JobState;

typedef struct {
    int job_id;
    pid_t pid;
    JobState state;
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

const char *state_name(JobState state)
{
    switch (state) {
        case RUNNING:
            return "Running";
        case STOPPED:
            return "Stopped";
        case COMPLETED:
            return "Completed";
        default:
    
```

Terminal Window Details:

- Terminal Title: arikela@DESKTOP-AMUSJME
- Editor: GNU nano 8.7.1
- File Name: task1_jobs.c *
- Bottom Status Bar:
 - ^G** Help
 - ^O** Write Out
 - ^F** Where Is
 - ^K** Cut
 - ^T** Execute
 - ^X** Exit
 - ^R** Read File
 - ^N** Replace
 - ^U** Paste
 - ^J** Justify

```
        default:
            return "Unknown";
    }
}

void add_job(pid_t pid)
{
    if (job_count >= MAX_JOBS)
        return;

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;
    jobs[job_count].state = RUNNING;

    job_count++;
}

void update_job_states()
{
    for (int i = 0; i < job_count; i++) {
        int status;

        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG | WUNTRACED | WCONTINUED
        );

        if (result == jobs[i].pid) {
            if (WIFEXITED(status) || WIFSIGNALED(status)) {
                jobs[i].state = COMPLETED;
            }
        }
    }
}
```

```

        if (result == jobs[i].pid) {

            if (WIFEXITED(status) || WIFSIGNALED(status)) {
                jobs[i].state = COMPLETED;
            }
            else if (WIFSTOPPED(status)) {
                jobs[i].state = STOPPED;
            }
            else if (WIFCONTINUED(status)) {
                jobs[i].state = RUNNING;
            }
        }
    }
}

void list_jobs()
{
    update_job_states();

    printf("\n===== ACTIVE JOBS =====\n");

    if (job_count == 0) {
        printf("No jobs found.\n");
        return;
    }

    printf("%-8s %-10s %-15s\n", "Job", "PID", "Status");
    printf("-----\n");

    for (int i = 0; i < job_count; i++) {
        printf("[%6d] %-10d %-15s\n",
            jobs[i].job_id,
            jobs[i].pid,

```

^G Help
 ^X Exit

^O Write Out
 ^R Read File

^F Where Is
 ^\ Replace

^K Cut
 ^U Paste

^T Execute
 ^J Justify

```

    for (int i = 0; i < job_count; i++) {
        printf("[%6d] %-10d %-15s\n",
            jobs[i].job_id,
            jobs[i].pid,
            state_name(jobs[i].state));
    }

    printf("=====\n");
}

int main()
{
    printf("TASK-1: Job Listing and Status Information\n");

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Child process started. PID = %d\n", getpid());

        for (int i = 1; i <= 10; i++) {
            printf("Child working... %d/10\n", i);
            sleep(1);
        }

        printf("Child process completed.\n");
        exit(0);
    }
}

```

[^]G Help
[^]X Exit

[^]O Write Out
[^]R Read File

[^]F Where Is
[^]\ Replace

[^]K Cut
[^]U Paste

[^]T Execute
[^]J Justify

```
        printf("Child process completed.\n");
        exit(0);
    }

    add_job(pid);

    for (int i = 0; i < 5; i++) {
        list_jobs();
        sleep(2);
    }

    waitpid(pid, NULL, 0);

    printf("\nFinal job status:\n");
    list_jobs();

    return 0;
}
```

Compile and run:

arike1a@DESKTOP-AMUSJME: x

+

✓

—

□

×

```
arike1a@DESKTOP-AMUSJME:~/week5prac$ gcc task1_jobs.c -o task1_j  
obs
```

```
arike1a@DESKTOP-AMUSJME:~/week5prac$ ./task1_jobs
```

TASK-1: Job Listing and Status Information

===== ACTIVE JOBS =====

Job	PID	Status
-----	-----	--------

[1	] 608	Running
----	-------	---------

Child process started. PID = 608

Child working... 1/10

Child working... 2/10

===== ACTIVE JOBS =====

Job	PID	Status
-----	-----	--------

[1	] 608	Running
----	-------	---------

Child working... 3/10

Child working... 4/10

===== ACTIVE JOBS =====

Job	PID	Status
-----	-----	--------

[1	] 608	Running
----	-------	---------

Child working... 5/10

Child working... 6/10

===== ACTIVE JOBS =====

Job	PID	Status
-----	-----	--------

[1	] 608	Running
----	-------	---------

Child working... 7/10

Child working... 8/10

```

===== ACTIVE JOBS =====
Job      PID      Status
-----
[1      ] 608      Running
=====
Child working... 7/10
Child working... 8/10

===== ACTIVE JOBS =====
Job      PID      Status
-----
[1      ] 608      Running
=====
Child working... 9/10
Child working... 10/10
Child process completed.

Final job status:

===== ACTIVE JOBS =====
Job      PID      Status
-----
[1      ] 608      Running
=====

```

Conclusion:

implemented a job table to maintain job ID, process ID and execution state. The program uses fork() to create a child process and waitpid() with WNOHANG, WUNTRACED and WCONTINUED to retrieve and update job status. A formatted job listing displays the job ID, PID and current state as Running, Stopped or Completed. The implementation was tested by executing a child process and observing its status during execution and after completion.

TASK-2: To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching

Ans:

```
arikel@DESKTOP-AMUSJME: × + v
GNU nano 8.7.1 task2_foreground.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

int main()
{
    printf("TASK-2: Foreground Job Switching\n");

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {

        setpgid(0, 0);

        printf("Child job started.\n");
        printf("Child PID: %d\n", getpid());
        printf("Child PGID: %d\n", getpgrp());

        for (int i = 1; i <= 5; i++) {
            printf("Foreground job running... %d/5\n", i);
            sleep(1);
        }

        printf("Foreground job completed.\n");
        exit(0);
    }
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify
```

```

        exit(0);
    }

    /*
     * Put child into its own process group.
     */
    setpgid(pid, pid);

    printf("Parent PID: %d\n", getpid());
    printf("Target Job PID: %d\n", pid);

    /*
     * Transfer terminal control to child process group.
     */
    printf("Transferring terminal control to child...\n");

    if (tcsetpgrp(STDIN_FILENO, pid) == -1) {
        perror("tcsetpgrp");
    }

    /*
     * Wait for foreground job.
     */
    int status;

    waitpid(pid, &status, WUNTRACED);

    /*
     * Restore terminal control to parent.
     */
    printf("Restoring terminal control to parent...\n");

    tcsetpgrp(STDIN_FILENO, getpgrp());

```

```

    tcsetpgrp(STDIN_FILENO, getpgrp());

    if (WIFEXITED(status)) {
        printf("Job completed normally.\n");
    }
    else if (WIFSIGNALED(status)) {
        printf("Job terminated by signal %d.\n",
            WTERMSIG(status));
    }
    else if (WIFSTOPPED(status)) {
        printf("Job stopped by signal %d.\n",
            WSTOPSIG(status));
    }

    printf("Foreground switching test completed.\n");

    return 0;
}
|

```

Compile and run:

```

arikel@DESKTOP-AMUSJME:~/week5prac$ nano task2_foreground.c
arikel@DESKTOP-AMUSJME:~/week5prac$ gcc task2_foreground.c -o task2_foreground
arikel@DESKTOP-AMUSJME:~/week5prac$ ./task2_foreground
TASK-2: Foreground Job Switching
Parent PID: 727
Target Job PID: 728
Transferring terminal control to child...
Child job started.
Child PID: 728
Child PGID: 728
Foreground job running... 1/5
Foreground job running... 2/5
Foreground job running... 3/5
Foreground job running... 4/5
Foreground job running... 5/5
Foreground job completed.
Restoring terminal control to parent...

[1]+  Stopped                  ./task2_foreground

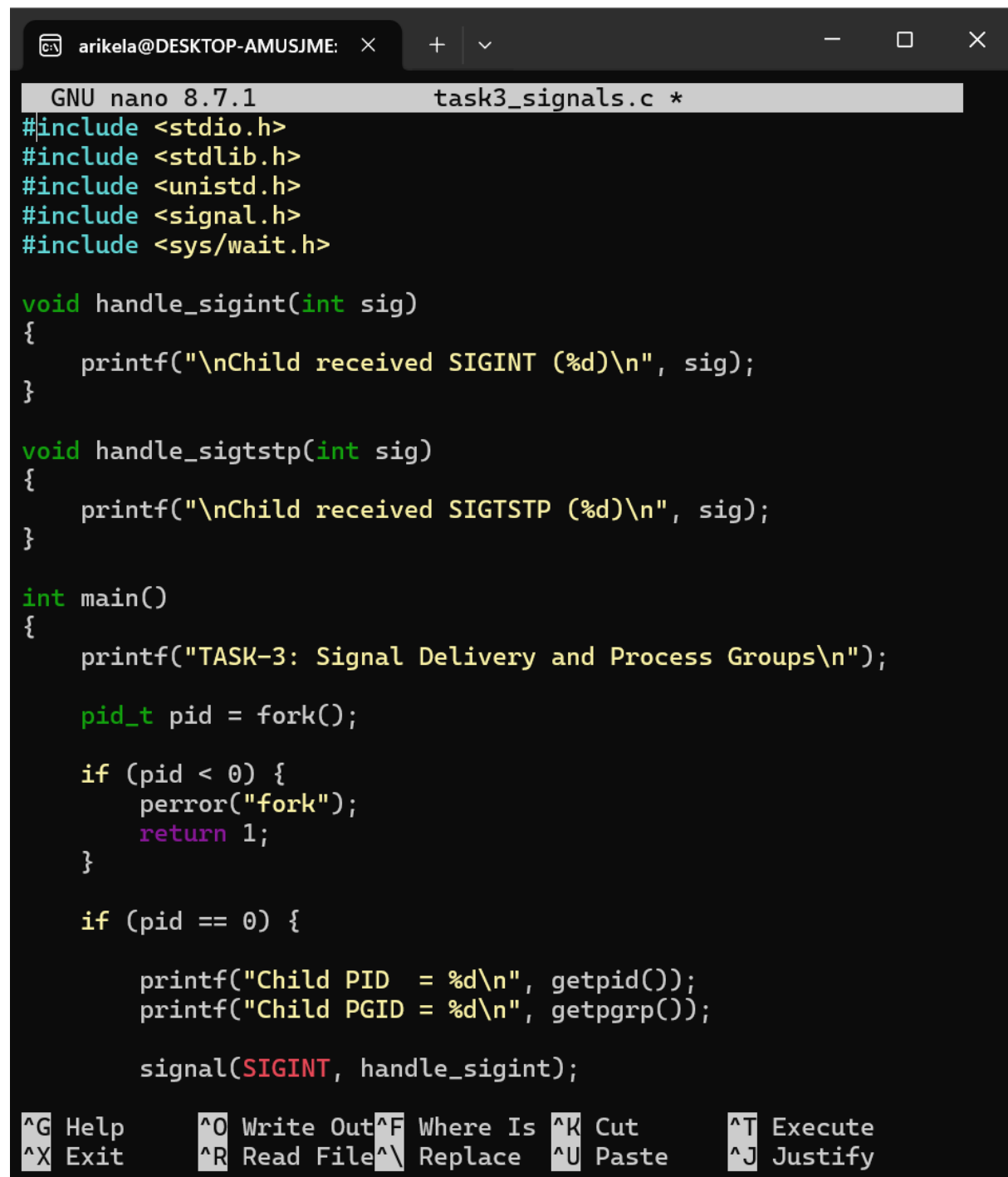
```

Conclusion: Implemented foreground job switching using process groups and terminal control. The target job is identified using its PID and assigned to a separate process group using `setpgid()`. Terminal control is transferred to the foreground process group using `tcsetpgrp()`. The parent waits for the job using `waitpid()` and restores terminal control after the job finishes or stops. The implementation was tested successfully using a foreground child process

TASK-3: To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior

Ans:

task3_signals.c:



```
GNU nano 8.7.1 task3_signals.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void handle_sigint(int sig)
{
    printf("\nChild received SIGINT (%d)\n", sig);
}

void handle_sigtstp(int sig)
{
    printf("\nChild received SIGTSTP (%d)\n", sig);
}

int main()
{
    printf("TASK-3: Signal Delivery and Process Groups\n");

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {

        printf("Child PID = %d\n", getpid());
        printf("Child PGID = %d\n", getpgrp());

        signal(SIGINT, handle_sigint);
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

```
signal(SIGINT, handle_sigint);
signal(SIGTSTP, handle_sigtstp);

/*
 * Create a signal mask.
 */
sigset_t mask;

sigemptyset(&mask);
sigaddset(&mask, SIGINT);

/*
 * Block SIGINT.
 */
sigprocmask(SIG_BLOCK, &mask, NULL);

printf("SIGINT is now blocked.\n");

for (int i = 1; i <= 5; i++) {
    printf("Child working... %d/5\n", i);
    sleep(1);
}

/*
 * Unblock SIGINT.
 */
sigprocmask(SIG_UNBLOCK, &mask, NULL);

printf("SIGINT unblocked.\n");

for (int i = 1; i <= 5; i++) {
    printf("Child continuing... %d/5\n", i);
    sleep(1);
}
```

```
        for (int i = 1; i <= 5; i++) {
            printf("Child continuing... %d/5\n", i);
            sleep(1);
        }

        exit(0);
    }

    printf("Parent PID = %d\n", getpid());
    printf("Sending SIGINT to child...\n");

    sleep(2);

    kill(pid, SIGINT);

    printf("SIGINT sent.\n");

    waitpid(pid, NULL, 0);

    printf("Child finished.\n");

    return 0;
}
```

Compile and run:

```

arikel@DESKTOP-AMUSJME:~/week5prac$ gcc task3_signals.c -o task3_signals
arikel@DESKTOP-AMUSJME:~/week5prac$ ./task3_signals
TASK-3: Signal Delivery and Process Groups
Parent PID = 810
Sending SIGINT to child...
Child PID = 811
Child PGID = 810
SIGINT is now blocked.
Child working... 1/5
Child working... 2/5
SIGINT sent.
Child working... 3/5
Child working... 4/5
Child working... 5/5

Child received SIGINT (2)
SIGINT unblocked.
Child continuing... 1/5
Child continuing... 2/5
Child continuing... 3/5
Child continuing... 4/5
Child continuing... 5/5
Child finished.
arikel@DESKTOP-AMUSJME:~/week5prac$

```

Conclusion:

Analyzed Linux signal delivery, signal masks and process groups. The program demonstrates sending signals using `kill()`, blocking signals using `sigprocmask()`, registering signal handlers and retrieving process-group information using `getpgrp()`. Terminal-related signals such as `SIGINT`, `SIGTSTP` and `SIGCONT` were studied and tested to understand their effect on processes.

TASK-4: To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verifr Execution, Test Background Recovery

Ans:

GNU nano 8.7.1

task4_background-recovery.c *

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

typedef enum {
    RUNNING,
    STOPPED,
    COMPLETED
} JobState;

typedef struct {
    int job_id;
    pid_t pid;
    JobState state;
} Job;

const char *state_name(JobState state)
{
    switch (state) {
        case RUNNING:
            return "Running";
        case STOPPED:
            return "Stopped";
        case COMPLETED:
            return "Completed";
        default:
            return "Unknown";
    }
}

void display_job(Job *job)
```

^G Help

^O Write Out

^F Where Is

^K Cut

^T Execute

^X Exit

^R Read File

^\\ Replace

^U Paste

^J Justify

```

void display_job(Job *job)
{
    printf("\n-----\n");
    printf("Job ID : %d\n", job->job_id);
    printf("PID      : %d\n", job->pid);
    printf("Status : %s\n", state_name(job->state));
    printf("-----\n");
}

int main()
{
    printf("TASK-4: Background Job Recovery\n");

    Job job;

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Background job started.\n");
        printf("PID = %d\n", getpid());

        for (int i = 1; i <= 10; i++) {
            printf("Job executing... %d/10\n", i);
            sleep(1);
        }

        printf("Background job completed.\n");
    }
}

```

```

        printf("Background job completed.\n");
        exit(0);
    }

    job.job_id = 1;
    job.pid = pid;
    job.state = RUNNING;

    display_job(&job);

    /*
     * Stop the job.
     */
    sleep(2);

    printf("\nStopping job using SIGSTOP...\n");

    kill(job.pid, SIGSTOP);

    int status;

    waitpid(job.pid, &status, WUNTRACED);

    if (WIFSTOPPED(status)) {
        job.state = STOPPED;
    }

    display_job(&job);

    /*
     * Resume the stopped job.

```

```

/*
 * Resume the stopped job.
 */
printf("\nSending SIGCONT to resume job...\n");

kill(job.pid, SIGCONT);

job.state = RUNNING;

display_job(&job);

/*
 * Wait until completion.
 */
waitpid(job.pid, &status, 0);

if (WIFEXITED(status) || WIFSIGNALED(status)) {
    job.state = COMPLETED;
}

display_job(&job);

printf("\nBackground recovery test completed.\n");

return 0;
}
|

```

Compile and run:

```
arikela@DESKTOP-AMUSJME: × + ▾  
arikela@DESKTOP-AMUSJME:~/week5prac$ gcc task4_background_recover  
y.c -o task4_background_recovery  
arikela@DESKTOP-AMUSJME:~/week5prac$ ./task4_background_recovery  
TASK-4: Background Job Recovery  
  
-----  
Job ID : 1  
PID    : 925  
Status : Running  
-----  
Background job started.  
PID = 925  
Job executing... 1/10  
Job executing... 2/10  
  
Stopping job using SIGSTOP...  
  
-----  
Job ID : 1  
PID    : 925  
Status : Stopped  
-----  
  
Sending SIGCONT to resume job...  
  
-----  
Job ID : 1  
PID    : 925  
Status : Running  
-----  
Job executing... 3/10  
Job executing... 4/10  
Job executing... 5/10  
Job executing... 6/10  
Job executing... 7/10  
Job executing... 8/10  
Job executing... 9/10
```

```
Job executing... 9/10
Job executing... 10/10
Background job completed.

-----
Job ID : 1
PID    : 925
Status : Completed
-----

Background recovery test completed.
arikel@DESKTOP-AMUSJME:~/week5prac$ |
```

Conclusion:

Implemented background job recovery using a maintained job table. A running process is stopped using SIGSTOP, and its state is updated to Stopped after detecting the state using waitpid() and WUNTRACED. The stopped process is resumed using SIGCONT, its state is changed back to Running, and the parent waits for completion. The final state is updated to Completed, verifying successful background recovery.